

آزمایشگاه سیستم عامل

پروژه سوم : زمانبندی در xv6

محمدامین رشید 810102454

سیدمحمدجواد شریفی 810102469

محمدصادق مهدوی پور 810102519

ج ۱) در xv6، بلوک کنترل پردازش (PCB) همان ساختار struct proc است که به صورت زیر است:

```
// Per-process state
struct proc {
    uint sz;                // Size of process memory (bytes)
    pde_t* pgdir;           // Page table
    char *kstack;           // Bottom of kernel stack for this process
    enum procstate state;   // Process state
    int pid;                // Process ID
    struct proc *parent;    // Parent process
    struct trapframe *tf;   // Trap frame for current syscall
    struct context *context; // swtch() here to run process
    void *chan;             // If non-zero, sleeping on chan
    int killed;             // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;       // Current directory
    char name[16];          // Process name (debugging)
};
```

همچنین وضعیت‌های تعریف شده برای هر پردازش، در ساختار زیر پیاده‌سازی شده است:

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

ساختار موجود در فصل ۳ کتاب نیز به صورت زیر است:

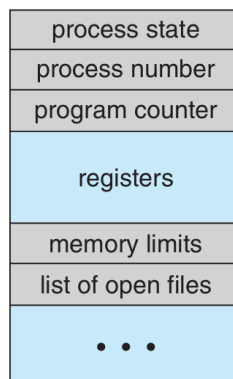


Figure 3.3 Process control block (PCB).

اکنون درباره تمام فیلدهای PCB در xv6 توضیح مختصری ارائه می‌دهیم و شباهت آنها با شکل کتاب را نیز مورد توجه قرار می‌دهیم:

۱- sz: اندازه فضای آدرس پردازش بر حسب بایت. همان memory limits در شکل کتاب است.

۲- pgdir: آدرس جدول صفحه پردازش.

۳- kstack: اشاره‌گر به ابتدای استک کرنل برای این پردازش.

۴- state: وضعیت فعلی پردازش (یکی از موارد موجود در enum procstate). همان process state در شکل کتاب است.

۵- pid: شماره پردازش. همان process number در شکل کتاب است.

۶- parent: اشاره‌گر به پردازش والد.

۷- tf: مخفف trap frame است. برای نگهداری رجیسترهای ذخیره شده هنگام ورود به کرنل است. رجیستر program counter و بقیه رجیسترها در آن نگهداری می‌شوند و به همین خاطر می‌توان گفت که معادل program counter و بخشی از registers در کتاب است.

۸- context: رجیسترهایی که برای تعویض متن ذخیره می‌شوند. ادامه بخش register در شکل کتاب است.

۹- chan: اطلاعات مربوط به حالت sleeping پردازش.

۱۰- killed: اطلاعات مربوط به حالت killed پردازش.

۱۱- file[]: آرایه‌ای از اشاره‌گرها به فایل‌های باز پردازش. همان list of open files در شکل کتاب است.

۱۲- cwd: دایرکتوری جاری پردازش.

۱۳- name[]: نام پردازش. (غالبا برای دیباگ به کار می‌رود).

بنابراین با توجه به اینکه تمام اجزاء PCB موجود در کتاب، در ساختار struct proc موجود در xv6 وجود دارند، می‌توانیم نتیجه بگیریم که میان ساختار معرفی شده PCB در کتاب با ساختار استفاده شده در xv6، شباهت‌ها کاملا مشخص است؛ و در واقع xv6 علاوه بر اینکه تمام بخش‌های PCB موجود در کتاب را در ساختار struct proc خود دارد، تعدادی فیلد اضافه‌تر نیز نگه می‌دارد.

ج ۲) همانطور که در بخش قبلی نیز عنوان شد، وضعیت‌های تعریف شده برای یک پردازش در xv6 به صورت زیر است:

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

حال هر یک از این وضعیت‌ها را به اختصار معرفی کرده و معادل آن در شکل ۱ را نام می‌بریم:

۱- UNSEND: این وضعیت معادلی در شکل ۱ ندارد. نشان‌دهنده آن است که این خانه از جدول پردازش‌ها فعلاً به هیچ پردازش اختصاص داده نشده است. (قبل از New یا بعد از Terminated)

۲- EMBRYO: این وضعیت معادل New در شکل است. نشان‌دهنده آن است که پردازش، تازه ساخته شده است.

۳- SLEEPING: این وضعیت معادل Waiting در شکل است. نشان‌دهنده آن است که پردازش منتظر رویدادی مثل I/O است و فعلاً پردازشی در پردازنده روی آن انجام نمی‌شود.

۴- RUNNABLE: این وضعیت معادل Ready در شکل است. نشان‌دهنده آن است که پردازش آماده اجراست و در صف آماده قرار دارد.

۵- RUNNING: این وضعیت معادل Running در شکل است. نشان‌دهنده آن است که پردازش در حال اجراست.

۶- ZOMBIE: این وضعیت معادل Terminated در شکل است. نشان‌دهنده آن است که اجرای پردازش تمام شده ولی منابع آن هنوز توسط والد جمع‌آوری نشده است.

ج ۳) تابع userinit برای اولین پردازش و تابع fork برای سایر پردازش‌ها، موجب گذر آنها از حالت New به Ready می‌شوند. این دو تابع بعد از صدا زدن تابع allocproc، وضعیت پردازش‌ای که تازه ساخته شده است را از حالت EMBRYO (معادل New) به حالت RUNNABLE (معادل Ready) می‌برند.

خود تابع allocproc نیز قبل از اجرای دو تابع مذکور، یک پردازش جدید در جدول پردازش‌ها که دارای وضعیت UNUSED است را به وضعیت EMBRYO می‌برد که در واقع همان مرحله ایجاد پردازش است.

ج ۴) در xv6 سقف تعداد پردازش‌ها برابر با مقدار NPROC است که در فایل param.h تعریف شده است:

```
C param.h > ...  
1 #define NPROC 64 // maximum number of processes
```

بنابراین سقف تعداد پردازش‌ها برابر ۶۴ است.

در صورتی که یک پردازش آنقدر fork کند که همه خانه‌های جدول پردازش‌ها پر شود، هنگام ایجاد یک پردازش جدید توسط تابع allocproc، دیگر هیچ پردازش‌ای با وضعیت UNUSED پیدا نمی‌شود و برای همین مقدار 0 را برمی‌گرداند. در این شرایط، واکنش کرنل این است که پردازش جدیدی نمی‌سازد و فقط مقدار برگشتی fork را برابر 1- قرار می‌دهد. برنامه سطح کاربر نیز مقدار بازگشتی تابع fork (که 1- بوده است) را دریافت می‌کند که نشان‌دهنده شکست در ایجاد پردازش فرزند است.

ج ۵) در ساختار xv6، یک جدول پردازش وجود دارد که بین همه CPU ها و تمام توابعی که وضعیت پردازش‌ها را عوض می‌کنند مشترک است. در تابع scheduler، روی همین جدول می‌گردیم و یک پردازش RUNNABLE را برای اجرا انتخاب می‌کنیم. با این توضیحات اگر هنگام اجرای scheduler، جدول را قفل نکنیم به مشکلاتی برخورد می‌کنیم:

در سیستم‌های چند هسته‌ای، اگر جدول را قفل نکنیم، ممکن است که چند هسته به طور همزمان پردازش‌ای را اجرا کنند. این اتفاق می‌تواند منجر به ایجاد یک تداخل بین منابع پردازش شود. همچنین می‌تواند باعث تولید داده‌های نادرست شود. به عنوان مثال ممکن است که به یک پردازش، چند شناسه متفاوت داده شود.

نیاز به قفل کردن جدول پردازش‌ها هنگام اجرای تابع scheduler، در سیستم‌های تک‌هسته‌ای نیز لازم است. زیرا روی یک هسته هم، کماکان وقفه‌ها و preemption های هسته وجود دارد. در نتیجه ممکن است دقیقاً در هنگامی که scheduler اجرا می‌شود، به خاطر رسیدن وقفه یا preemption، تغییری در وضعیت پردازش‌ها صورت بگیرد و بدین ترتیب منجر به بروز اشتباه در نتیجه تابع scheduler شود.

بنابراین لازم است که در ابتدای هر دور حلقه تابع scheduler، با استفاده از acquire(&ptable.lock)، جدول را قفل کرده و در انتهای حلقه با استفاده از release(&ptable.lock)، قفل را برداریم.

ج ۶) در ساختار xv6 (چه در حالت تک‌هسته‌ای و چه در حالت چند هسته‌ای) فقط یک جدول پردازش داریم و هر تغییری در وضعیت یکی از پردازش‌ها، روی همین جدول انجام می‌شود. با توجه به پاسخ سوال قبل، می‌دانیم که در ابتدای هر دور حلقه تابع scheduler، این جدول قفل می‌شود و وقتی یک هسته، جدول را قفل می‌کند، این قفل فقط دست همان هسته است و هیچ هسته‌ی دیگری نمی‌تواند حالت پردازش‌ای را به RUNNABLE تغییر دهد. بنابراین وقتی یک پردازش آماده به اجرا می‌شود، این تغییر وضعیت فقط بعد از آزاد شدن قفل اعمال می‌شود؛ پس آن پردازش در iteration بعدی امکان schedule شدن دارد نه در iteration جاری که جدول در حال پیمایش است.

کد تابع scheduler که در فایل proc.c وجود دارد به صورت زیر است:

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // Enable interrupts on this processor.
        sti();

        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;

            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            c->proc = p;
            switchvm(p);
            p->state = RUNNING;

            swtch(&(c->scheduler), p->context);
            switchkvm();

            // Process is done running for now.
            // It should have changed its p->state before coming back.
            c->proc = 0;
        }
        release(&ptable.lock);
    }
}

```

ساختار context در proc.h 5 تا رجیستر دارد. از این رجیستر ها برای context switch استفاده میشود.

رجیستر edi : (extended destination index) رجیستر مقصد که به صورت عمومی برای کار های حافظه ای استفاده میشود.

رجیستر esi : (extended source index) رجیستر عمومی دیگر برای کار های تکراری روی حافظه

رجیستر ebx : (extended base register) یک رجیستر همه کاره برای ذخیره سازی دیتا های موجود در یک پردازنده

رجیستر ebp : (extended base pointer) رجیستری که شروع فریم یک تابع را در استک نشان میدهد

رجیستر eip : (extended instruction pointer) رجیستری که آدرس دستور بعدی را که باید اجرا شود را نگه میدارد.

```
32 // but it is on the stack an
33 struct context {
34     uint edi;
35     uint esi;
36     uint ebx;
37     uint ebp;
38     uint eip;
39 };
40
```

ج ۸) همانطور که گفتم رجیستر eip وظیفه نگه داری آدرس دستور بعد را دارد که همان مقدار program counter است .

```
swtch(&(c->scheduler), best->context);
```

با دستور بالا در تابع scheduler ، کد اسمبلی زیر در نهایت اجرا میشود. در این کد قصد داریم ابتدا مقادیر رجیستر کنونی (c->scheduler) را ذخیره کنیم و رجیستر های best->context را روی cpu لود کنیم.

البته این رجیستر مستقیم پوش و پاپ در دستورات اسمبلی زیر لازم نیست بشود و این رجیستر از به عنوان `base` در نظر گرفته میشود و در آخر هنگام `ret` ، محتوای همین `eip` بازیابی میشود و در ابتدا برای ذخیره کردن هم در استک توسط `cpu` از همان فراخوانی `switch` این مقدار ذخیره میشود.

```
Makefile U  C proc.c M  C proc.h  ASM switch.S x  C console.c
LAB3 > xv6 > ASM switch.S
1  # Context switch
2  #
3  # void switch(struct context **old, struct context *new);
4  #
5  # Save the current registers on the stack, creating
6  # a struct context, and save its address in *old.
7  # Switch stacks to new and pop previously-saved registers.
8
9  .globl switch
10 switch:
11     movl 4(%esp), %eax
12     movl 8(%esp), %edx
13
14     # Save old callee-saved registers
15     pushl %ebp
16     pushl %ebx
17     pushl %esi
18     pushl %edi
19
20     # Switch stacks
21     movl %esp, (%eax)
22     movl %edx, %esp
23
24     # Load new callee-saved registers
25     popl %edi
26     popl %esi
27     popl %ebx
28     popl %ebp
29     ret
30
```

ج9) اگر `interrupt` فعال نباشد ، به مشکل اساسی برمیخوریم:

قابلیت `preemptive` بودن از دست میرود و نمیتونیم طبق الگوریتمی که میخواهیم (مثلا `Round Robin`) `cpu` را از یک پردازش بگیریم زیرا دیگر تایمر درست کار نمیکند (تایمر از طریق وقفه سخت افزاری عمل میکند) و به نوعی الگوریتم به صورت `fifo` میشود و ممکن است یک پردازش `cpu` را به صورت انحصاری اشغال کند.

ج10) این مقدار طبق کتاب `xv6 10` میلی ثانیه است. یعنی هر `10` میلی ثانیه وقفه تایمر صدا زده میشود.

با انجام کار زیر ، تقریبا در هر ثانیه 100 بار عبارت ++ticks چاپ میشود.

```
case T_IRQ0 + IRQ_TIMER:
    if(cpuid() == 0){
        acquire(&tickslock);
        ticks++;
        cprintf("ticks++");

        wakeup(&ticks);
        release(&tickslock);
    }
    lapiceoi();
    break;
case T_IRQ0 + IRQ_IDE:
    ideintr();
```

هم چنین در تابع زیر:

```
54 void
55 lapicinit(void)
56 {
57     if(!lapic)
58         return;
59
60     // Enable local APIC; set spurious interrupt vector.
61     lapicw(SVR, ENABLE | (T_IRQ0 + IRQ_SPURIOUS));
62
63     // The timer repeatedly counts down at bus frequency
64     // from lapic[TICR] and then issues an interrupt.
65     // If xv6 cared more about precise timekeeping,
66     // TICR would be calibrated using an external time source.
67     lapicw(TDCR, X1);
68     lapicw(TIMER, PERIODIC | (T_IRQ0 + IRQ_TIMER));
69     lapicw(TICR, 10000000);
70
71     // Disable logical interrupt lines.
72     lapicw(LINT0, MASKED);
73     lapicw(LINT1, MASKED);
74
75     // Disable performance counter overflow interrupts
76     // on machines that provide that interrupt entry.
77     if(((lapic[VER]>>16) & 0xFF) >= 4)
78         lapicw(PCINT, MASKED);
79
80     // Map error interrupt to IRQ_ERROR.
81     lapicw(ERROR, T_IRQ0 + IRQ_ERROR);
82
83     // Clear error status register (requires back-to-back writes).
84     lapicw(ESR, 0);
85     lapicw(ESR, 0);
86
87     // Ack any outstanding interrupts.
88     lapicw(EOI, 0);
89
90     // Send an Init Level De-Assert to synchronise arbitration ID's.
91     lapicw(ICRHI, 0);
92     lapicw(ICRLO, BCAST | INIT | LEVEL);
93     while(lapic[ICRLO] < BCAST);
```

برای ایجاد وقفه ، سخت افزار یک کانتر 10000000 می‌شمرد و سپس یک وقفه صادر میکند که همان وقفه تایمر است. با دانستن فرکانس تقریبی این شمارنده ، میتوان بازهم به جواب 10 میلی ثانیه رسید.

ج11) در تکه کد زیر در تابع trap ، علاوه بر زیاد تر شدن ticks در اثر وقفه تایمری ، تابع yield نیز صدا زده میشود.

```
94
95 // Force process to give up CPU on clock tick.
96 // If interrupts were on while locks held, would need to check nlock.
97 if(myproc() && myproc()->state == RUNNING &&
98     tf->trapno == T_IRQ0+IRQ_TIMER)
99     yield();
10
```

که در تابع yield وضعیت را از RUNNING به RUNNABLE تغییر میدهد(طبق شکل 1) و سپس sched را صدا میزند تا زمان بندی و contex switch انجام شود.

```
411 // there's no process.
412 void sched(void)
413 {
414     int intena;
415     struct proc *p = myproc();
416
417     if (!holding(&ptable.lock))
418         panic("sched ptable.lock");
419     if (mycpu()->ncli != 1)
420         panic("sched locks");
421     if (p->state == RUNNING)
422         panic("sched running");
423     if (readeflags() & FL_IF)
424         panic("sched interruptible");
425     intena = mycpu()->intena;
426     swtch(&p->context, mycpu()->scheduler);
427     mycpu()->intena = intena;
428 }
429
430 // Give up the CPU for one scheduling round.
431 void yield(void)
432 {
433     acquire(&ptable.lock); // DOC: yieldlock
434     myproc()->state = RUNNABLE;
435     sched();
436     release(&ptable.lock);
437 }
438
```

ج12)

از آنجایی که این تابع yield توسط وقفه تایمری صدا زده میشه

و وقفه تایمری در هر 10 میلی ثانیه رخ میده ، پس کوانتوم زمانی در xv6 به صورت دیفالت همان 10ms است و پردازش پس از 10 میلی ثانیه پردازش را پس میده(حداکثر 10 میلی ثانیه ممکن است کارش زودتر تمام شود (

ج13) تابع wait وقتی اجرا میشود ، در یک لوپ می افتاد که چک میکند و 3 حالت مختلف ممکن است رخ دهد:

1) پردازش ،فرزند zombie داشته باشد : در این حالت پردازش را از ptable حذف کرده و منابع آن را آزاد میکند و pid آن را return میکند.

```
int wait(void)
{
    struct proc *p;
    int havekids, pid;
    struct proc *curproc = myproc();

    acquire(&ptable.lock);
    for (;;)
    {
        // Scan through table looking for exited children
        havekids = 0;
        for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        {
            if (p->parent != curproc)
                continue;
            havekids = 1;
            if (p->state == ZOMBIE)
            {
                // Found one.
                pid = p->pid;
                kfree(p->kstack);
                p->kstack = 0;
                freevm(p->pgdir);
                p->pid = 0;
                p->parent = 0;
                p->name[0] = 0;
                p->killed = 0;
                p->state = UNUSED;
                release(&ptable.lock);
                return pid;
            }
        }
    }
}
```

2) پردازش فرزند نداشته باشد یا خودش kill شده باشد: 1- را return میکند.

```
// No point waiting if we don't have any children.  
if (!havekids || curproc->killed)  
{  
    release(&ptable.lock);  
    return -1;  
}
```

(3) پردازش ، فرزند داشته باشد ولی هنوز zombie نشده است و در حال اجراست.

در این حالت باید sleep صدا زده شود تا صبر کند فرزندش خودش هنگام exit این پراسس را بیدار کند

```
// Wait for children to exit. (See wakeup1 call in proc_exit.)  
sleep(curproc, &ptable.lock); // DOC: wait-sleep  
}
```

به تابع sleep یک chan و یک lock می‌دهیم . قفل را همان p.table.lock می‌دهیم و chan را خود پراسس تا پردازش روی کانال خودش به خواب برود. می‌بینیم که در این تابع shed فراخوانی شده است تا cpu از این پردازش گرفته شود و تا زمانی که فرزند wakeup1 را فراخوانی نکند ، state آن به صورت SLEEPING می‌ماند.

```

61 // Reacquires lock when awakened.
62 void sleep(void *chan, struct spinlock *lk)
63 {
64     struct proc *p = myproc();
65
66     if (p == 0)
67         panic("sleep");
68
69     if (lk == 0)
70         panic("sleep without lk");
71
72     // Must acquire ptable.lock in order to
73     // change p->state and then call sched.
74     // Once we hold ptable.lock, we can be
75     // guaranteed that we won't miss any wakeup
76     // (wakeup runs with ptable.lock locked),
77     // so it's okay to release lk.
78     if (lk != &ptable.lock)
79     {
80         // DOC: sleeplock0
81         acquire(&ptable.lock); // DOC: sleeplock1
82         release(lk);
83     }
84     // Go to sleep.
85     p->chan = chan;
86     p->state = SLEEPING;
87
88     sched();
89
90     // Tidy up.
91     p->chan = 0;
92
93     // Reacquire original lock.
94     if (lk != &ptable.lock)
95     {
96         // DOC: sleeplock2
97         release(&ptable.lock);
98         acquire(lk);
99     }
100 }

```

Wakeup1 در تابع `exit` در یک پردازش فراخوانی میشود تا پردازش هنگام `exit` شدن، والد خود را مطلع کند و از `sleep` در بیاورد تا بتواند در مرحله بعد که فرزندانش را چک میکند، فرزندی که اکنون ZOMBIE شده است را از `ptable` حذف و منابعش را آزاد کند.

اگر یک پردازش چندین فرزند داشته باشد، باید برای همه آن ها روند بالا طی شود تا بتواند از `wait` خارج شود.

ج14) تابع `sleep` تنها کاربردش در `wait` نیست. در بخش `console.c` دیدیم که موقعی که کاربر اینتر را میزد فرخوانی سیستمی `wakeup(&input.r)` انجام میشد تا `process` ای که در اینجا روی کانال `input.r` خوابیده است بیدار شود و در این حالت از حلقه `while` بیرون می آید زیرا بافر خالی نیست و به بخش بعد می رود

و از بافر میخواند (از input.r تا input.w) و سپس برابر میشود و دوباره به sleep میرود تا دوباره فراخوانی wakeup انجام شود.

```
LAB3 > xv6 > C console.c > ...
1056
1057 int consoleread(struct inode *ip, char *dst, int n)
1058 {
1059     uint target;
1060     int c;
1061
1062     iunlock(ip);
1063     target = n;
1064     acquire(&cons.lock);
1065     while (n > 0)
1066     {
1067         while (input.r == input.w)
1068         {
1069             if (myproc()->killed)
1070             {
1071                 release(&cons.lock);
1072                 ilock(ip);
1073                 return -1;
1074             }
1075             sleep(&input.r, &cons.lock);
1076         }
1077         c = input.buf[input.r++ % INPUT_BUF];
1078         if (c == C('D'))
1079         { // EOF
1080             if (n < target)
1081             {
1082                 // Save ^D for next time, to make sure
1083                 // caller gets a 0-byte result.
1084                 input.r--;
1085             }
1086             break;
1087         }
1088         *dst++ = c;
1089         --n;
1090         if (c == '\n')
1091             break;
1092     }
1093     release(&cons.lock);
1094     ilock(ip);
1095
1096     return target - n;
1097 }
1098
```

ج15) در xv6 تابع () wakeup وظیفه آگاه سازی پردازش ای را دارد که منتظر آن بوده است.

ورودی این تابع از تایپ *chan است. یعنی میگیریم کسی که در کانال chan خوابیده است، بیدار شود.

پس از قفل کردن ptable همانطور که می بینیم تمامی پردازش هایی که در حالت SLEEPING هستند و روی کانالی که ورودی دادیم هستند، RUNNABLE میشوند.

```

499
500 // PAGEBREAK!
501 // Wake up all processes sleeping on chan.
502 // The ptable lock must be held.
503 static void
504 wakeup1(void *chan)
505 {
506     struct proc *p;
507
508     for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
509         if (p->state == SLEEPING && p->chan == chan)
510             p->state = RUNNABLE;
511 }
512
513 // Wake up all processes sleeping on chan.
514 void wakeup(void *chan)
515 {
516     acquire(&ptable.lock);
517     wakeup1(chan);
518     release(&ptable.lock);
519 }
520

```

ج 16) پردازش orphan پردازش یتیم است . یعنی پردازش ای که به حالت zombie در آمده ولی والدش خاتمه یافته و دیگر والد نمیتواند منابع فرزند را آزاد کند.

در xv6، یک پردازش هنگام خاتمه (در تابع exit) ، بعد از اینکه فراخوانی wakeup1 را انجام داد(به عنوان نقش فرزند) ، باید چک کند که آیا این پردازش فرزندی دارد(به عنوان نقش والد). در این حالت باید سرپرستی فرزندان به پراسس init سپرده شود و اگر یکی از فرزندان ZOMBIE بوده است، اکنون init را بیدار میکنیم تا بیاید و فرزند جدید ZOMBIE اش را خاتمه دهد. این فرآیند به نوعی به صورت دوره ای تکرار میشود به صورتی که هر بار که یک پردازش قرار باشد بمیرد ، این موضوع چک میشود تا یتیم ها هم بتوانند به درستی پاکسازی شوند و منابع از آن ها گرفته شود.

```
3 // Parent might be sleeping in wait().
4 wakeup1(curproc->parent);
5
6 // Pass abandoned children to init.
7 for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
8 {
9     if (p->parent == curproc)
10    {
11        p->parent = initproc;
12        if (p->state == ZOMBIE)
13            wakeup1(initproc);
14    }
15 }
16 }
```

بخش ۴) زمان‌بندی هسته‌های ناهمگون

در ابتدای این بخش، فیلد جدیدی به نام `core_type` به ساختار `cpu` (در فایل `proc.h`) اضافه می‌کنیم:

```
enum core_type {
    ECORE = 0,
    PCORE = 1
};
// Per-CPU state
struct cpu {
    uchar apicid;
    struct context *scheduler;
    struct taskstate ts;
    struct segdesc gdt[NSEGS];
    volatile uint started;
    int ncli;
    int intena;
    struct proc *proc;

    enum core_type type;
};
```

سپس `core_type` یک `cpu` را در هنگام پر کردن آرایه `cpus` در تابع `mpinit` (در فایل `mp.c`)، بر اساس `id` آن مشخص کنیم:

```
for(p=(uchar*)(conf+1), e=(uchar*)conf+conf->length; p<e; ){
    switch(*p){
    case MPPROC:
        proc = (struct mpproc*)p;
        if(ncpu < NCPU) {
            cpus[ncpu].apicid = proc->apicid; // apicid may differ from ncpu
            cpus[ncpu].type = (ncpu % 2 == 0) ? ECORE : PCORE; // add for LAB3
            ncpu++;
        }
    }
```

ج ۱۷) همانطور که دیدیم ساختار `cpu` در `proc.h` است که هر فیلد آن را در صفحه بعد توضیح داده ام :

```
5 // Per-CPU state
6 struct cpu {
7     uchar apicid; // Local APIC ID
8     struct context *scheduler; // switch() here to enter scheduler
9     struct taskstate ts; // Used by x86 to find stack for interrupt
10    struct segdesc gdt[NSEGS]; // x86 global descriptor table
11    volatile uint started; // Has the CPU started?
12    int ncli; // Depth of pushcli nesting.
13    int intena; // Were interrupts enabled before pushcli?
14    struct proc *proc; // The process running on this cpu or null
15
16    enum core_type type; // add for LAB3
17};
```

uchar Apicid : شناسه محلی یک cpu است تا در حالت های چند پردازنده ای بتوانیم بین وقفه های cpu ... تمایز قایل شویم. در حالت تک پردازنده ای این برای آن پردازنده 0 است.

Struct contex * scheduler : این ساختار شامل 5 تا رجیستر است . این ذخیره میشود تا اگر کار یک پردازنده روی این cpu تمام شد ، (تمام شدن کوانتوم یا exit) cpu بداند که به کدام محتوای رجیستری باید مراجعه کند و مقادیر این رجیستر ها از طریق تابع scheduler تعیین میشود.

Struct taskstate ts : مهم ترین فیلد این بخش esp0 است که آدرس بالای استک kernel را نشان میدهد تا هنگام سیستم کال و وقفه به درستی به آدرس درست در کرنل برود.

```
106 // Task state segment format
107 struct taskstate {
108     uint link;           // Old ts selector
109     uint esp0;           // Stack pointers and segment sele
110     ushort ss0;          // after an increase in privileg
111     ushort padding1;
112     uint *esp1;
113     ushort ss1;
114     ushort padding2;
115     uint *esp2;
116     ushort ss2;
117     ushort padding3;
118     void *cr3;           // Page directory base
119     uint *eip;           // Saved state from last task swit
120     uint eflags;
121     uint eax;            // More saved state (registers)
122     uint ecx;
123     uint edx;
124     uint ebx;
125     uint *esp;
126     uint *ebp;
127     uint esi;
128     uint edi;
129     ushort es;           // Even more saved state (segment
130     ushort padding4;
131     ushort cs;
132     ushort padding5;
133     ushort ss;
134     ushort padding6;
135     ushort ds;
136     ushort padding7;
137     ushort fs;
138     ushort padding8;
139     ushort gs;
140     ushort padding9;
141     ushort ldt;
142     ushort padding10;
143     ushort t;            // Trap on task switch
```

Segdesc gdt[] : جدول توصیف گر های سگمنت است که شامل مواردی میشود که نشان دهد هر سگمنت در چه آدرسی قرار دارد مثلا کد کرنل کجاست یا دیتا یوزر کجاست

```

26 struct segdesc {
27     uint lim_15_0 : 16; // Low bits of segment limit
28     uint base_15_0 : 16; // Low bits of segment base address
29     uint base_23_16 : 8; // Middle bits of segment base address
30     uint type : 4; // Segment type (see STS_constants)
31     uint s : 1; // 0 = system, 1 = application
32     uint dpl : 2; // Descriptor Privilege Level
33     uint p : 1; // Present
34     uint lim_19_16 : 4; // High bits of segment limit
35     uint avl : 1; // Unused (available for software use)
36     uint rsv1 : 1; // Reserved
37     uint db : 1; // 0 = 16-bit segment, 1 = 32-bit segment
38     uint g : 1; // Granularity: limit scaled by 4K when set
39     uint base_31_24 : 8; // High bits of segment base address
40 };
41

```

Started : cpu هنگام بوت این مقدار را برای خودش 1 میکند تا بداند شروع به کار کرده است.

Ncli : عمق تعداد دفعات pushcli را نشان میدهد. یعنی تعداد دفعاتی که وقفه غیر فعال شده است پس به همین تعداد نیاز هست تا popcli فراخوانی شود تا وقفه فعال شود.

Intena : وضعیت فعال بودن یا غیرفعال بودن وقفه در قبل آخرین pushcli را نشان میدهد.

***proc :** اشاره گر به پردازشگر ای که اکنون در حال اجرا روی این cpu می باشد.

بخش 4.1) هسته های E با شماره CPUID زوج : زمان بند نوبت گردشی با کوانتوم زمانی

ابتدا به struct یک پردازش ، فیلد ticks رو اضافه میکنیم تا به کمک این متغیر بتوانیم گردش زمانی کوانتومی رو هندل بکنیم و بدانیم هر پردازش ، چند تیک است که تا الان اجرا شده است.

برای مثال اگر تیک یک پردازش 2 باشد ، بدین معنی است که در لحظه 20 تا 30 میلی ثانیه است. هر tick در xv6 10 میلی ثانیه است.

```
3 // Per-process state
4 struct proc {
5     uint sz; // Size of process memory (bytes)
6     pde_t* pgdir; // Page table
7     char *kstack; // Bottom of kernel stack for this process
8     enum procstate state; // Process state
9     int pid; // Process ID
10    struct proc *parent; // Parent process
11    struct trapframe *tf; // Trap frame for current syscall
12    struct context *context; // swch() here to run process
13    void *chan; // If non-zero, sleeping on chan
14    int killed; // If non-zero, have been killed
15    struct file *ofile[NOFILE]; // Open files
16    struct inode *cwd; // Current directory
17    char name[16]; // Process name (debugging)
18    int priority;
19
20    uint create_time; // add for FCFS
21    int host_cpu; // add for moving between queues
22    int ticks; //add for Round Robin
23 };
24
```

به ساختار cpu ، فیلد last_RR_proc را نیز اضافه میکنیم تا بدانیم که ایندکس آخرین پردازش ای که روی این پردازنده در حال اجرا بوده یا همین الان در حال اجراست ، چند است.(این را اضافه میکنیم تا گردش به درستی صورت گیرد و سری بعد که این پردازنده بخواهد زمان بندی شود ، از پردازش بعدی در ptable شروع به لوپ زدن کنیم)

```
5 // Per-CPU state
6 struct cpu {
7     uchar apicid; // Local APIC ID
8     struct context *scheduler; // swch() here to enter scheduler
9     struct taskstate ts; // Used by x86 to find current task
10    struct segdesc gdt[NSEGS]; // x86 global descriptor table
11    volatile uint started; // Has the CPU started?
12    int ncli; // Depth of pushcli nested calls
13    int intena; // Were interrupts enabled before?
14    struct proc *proc; // The process running on this CPU
15    int last_RR_proc; //add for RR
16    enum core_type type; // add for LAB3
17 };
18
```

مقدار اولیه این فیلد را در فایل mp.c برابر منفی 1 قرار میدهیم تا دفعه اول از اول ptable شروع کند.

```
proc = (struct mpproc*)p;
if(ncpu < NCPU) {
    cpus[ncpu].apicid = proc->apicid; // apicid may differ from ncpu
    cpus[ncpu].type = (ncpu % 2 == 0) ? ECORE : PCORE; // add for LAB3
    cpus->last_RR_proc = -1;
    ncpu++;
}
```

سپس تغییر زیر را در تابع زمان بند اعمال میکنیم تا اگر هسته E باشد، زمان بند مخصوص به خودش اجرا شود.

```
437 void
438 scheduler(void)
439 {
440     struct proc *p;
441     struct cpu *c = mycpu();
442     c->proc = 0;
443     int cpu_id = cpuid();
444
445     for (;;)
446     {
447         // Enable interrupts on this processor.
448         sti();
449
450         // Loop over process table looking for process to run.
451         acquire(&ptable.lock);
452
453         if (cpu_id % 2 == 0)
454         {
455             RR_scheduler();
456             release(&ptable.lock);
457             continue;
458         }
459     }
460
461     // start of FCFS
```

در خود الگوریتم، از last_RR_proc شروع میکنیم تا زمانی که یک دور کامل نزدیم شروع به چک کردن پراسس های Ready میکنیم.(Runnable)

در صورت یافتن پردازش مناسب، عملیات contex switch را انجام میدهیم و dispatch انجام میشود.

```

331 }
332 void RR_scheduler(void)
333 {
334     struct proc *p;
335     struct cpu *c = mycpu();
336     c->proc = 0;
337     for (int i = 0; i < NPROC; i++)
338     {
339         int idx = (mycpu()->last_RR_proc + 1 + i) % NPROC;
340
341
342         p = &ptable.proc[idx];
343         if (p->state == RUNNABLE)
344         {
345             c->proc = p;
346             c->last_RR_proc = idx;
347
348             switchvm(p);
349             p->state = RUNNING;
350             p->ticks = 0;
351             swtch(&(c->scheduler), p->context);
352             switchkvm();
353             c->proc = 0;
354             return;
355         }
356     }
357 }

```

در فایل trap.c، ابتدا صدا زدن yield که به اجبار cpu را از پردازش می گیرد، را برای هسته های زوج و فرد جدا میکنیم.

در حالت زوج، به ازای هر تیک دیگر زمان بندی انجام نمیدهیم، تیک های یک پراسس که به صورت جدا اضافه کرده بودیم، باید به حد مطلوب برسد(که ما در اینجا با قرار دادن 3، کوانتوم 30 میلی ثانیه ای ایجاد کردیم) تا تابع yield صدا زده شود. در غیر این صورت فقط tick یکی افزایش پیدا میکند.

```

// If interrupts were on while locks held, would need to check tlock.
if (myproc() && (cpuid() % 2 == 0) && myproc()->state == RUNNING && tf->trapno == T_IRQ0 + IRQ_TIMER)
{
    cprintf("Running on CPU   cpu%d pid=%d ticks=%d time=%d\n", cpuid(), myproc()->pid, myproc()->ticks, myproc()->ticks*10);
    myproc()->ticks++;

    if (myproc()->ticks == 3)
    {
        cprintf("Finish Quantum ! cpu%d pid=%d ticks=%d time=%d\n", cpuid(), myproc()->pid, myproc()->ticks, myproc()->ticks*10);
        myproc()->ticks = 0;

        yield();
    }
}
else if (myproc() && myproc()->state == RUNNING &&
        tf->trapno == T_IRQ0 + IRQ_TIMER)
    yield();

```

برنامه سطح کاربر:

در این برنامه پردازش والد ، 4 تا fork انجام میدهد. هر یک از 4 فرزند کار محاسباتی سنگینی انجام میدهند.

```
LAB3 > xv6 > C rrtest.c > ...
1
2 #include "types.h"
3 #include "stat.h"
4 #include "user.h"
5
6 void busy_print(int id) {
7     volatile unsigned long i = 0;
8
9     while (1) {
10
11         for (unsigned long k = 0; k < 1000000UL; k++)
12             i++;
13     }
14 }
15
16 int main(void) {
17     for (int i = 1; i <= 4; i++) {
18         if (fork() == 0) {
19             busy_print(i);
20             exit();
21         }
22     }
23
24     for (;;)
25         asm volatile("");
26
27     exit();
28 }
29
```

با قرار دادن پرینت مناسب در trap خروجی زیر را در حالت تک پردازنده ای مشاهده میکنیم:

همانطور که از شکل مشخص است ، فقط 0 cpu را داریم که زمان بندی روی آن انجام میشود. پردازش ها را به ترتیب اندیس اجرا میکند یعنی ابتدا 4 بعد 5 بعد 6 بعد 7 و دوباره برمیگردد و از 4 شروع میکند. به هر کدام از پردازش ها 30 میلی ثانیه حداکثر فرصت میدهد ، همانطور که معلوم است از 0 شروع میشود و در لحظه 30 میلی ثانیه ، کوانتوم خاتمه پیدا میکند. و می بینیم که الگوریتم به درستی کار میکند.

```
M Makefile LAB2/... U M Makefile LAB3/... C proc.c C balance_test.c C rrtest.c X C mp.c C trap.c M C proc.h M
LAB3 > xv6 > C rrtest.c > main(void)
1
2 #include "types.h"

PROBLEMS OUTPUT TERMINAL PORTS

Running on CPU cpu0 pid=4 ticks=1 time=10
Running on CPU cpu0 pid=4 ticks=2 time=20
Finish Quantum ! cpu0 pid=4 ticks=3 time=30
Running on CPU cpu0 pid=5 ticks=0 time=0
Running on CPU cpu0 pid=5 ticks=1 time=10
Running on CPU cpu0 pid=5 ticks=2 time=20
Finish Quantum ! cpu0 pid=5 ticks=3 time=30
Running on CPU cpu0 pid=6 ticks=0 time=0
Running on CPU cpu0 pid=6 ticks=1 time=10
Running on CPU cpu0 pid=6 ticks=2 time=20
Finish Quantum ! cpu0 pid=6 ticks=3 time=30
Running on CPU cpu0 pid=7 ticks=0 time=0
Running on CPU cpu0 pid=7 ticks=1 time=10
Running on CPU cpu0 pid=7 ticks=2 time=20
Finish Quantum ! cpu0 pid=7 ticks=3 time=30
Running on CPU cpu0 pid=3 ticks=0 time=0
Running on CPU cpu0 pid=3 ticks=1 time=10
Running on CPU cpu0 pid=3 ticks=2 time=20
Finish Quantum ! cpu0 pid=3 ticks=3 time=30
Running on CPU cpu0 pid=4 ticks=0 time=0
Running on CPU cpu0 pid=4 ticks=1 time=10
Running on CPU cpu0 pid=4 ticks=2 time=20
Finish Quantum ! cpu0 pid=4 ticks=3 time=30
Running on CPU cpu0 pid=5 ticks=0 time=0
Running on CPU cpu0 pid=5 ticks=1 time=10
Running on CPU cpu0 pid=5 ticks=2 time=20
Finish Quantum ! cpu0 pid=5 ticks=3 time=30
Running on CPU cpu0 pid=6 ticks=0 time=0
Running on CPU cpu0 pid=6 ticks=1 time=10
Running on CPU cpu0 pid=6 ticks=2 time=20
Finish Quantum ! cpu0 pid=6 ticks=3 time=30
Running on CPU cpu0 pid=7 ticks=0 time=0
Running on CPU cpu0 pid=7 ticks=1 time=10
Running on CPU cpu0 pid=7 ticks=2 time=20
Finish Quantum ! cpu0 pid=7 ticks=3 time=30
Running on CPU cpu0 pid=3 ticks=0 time=0
Running on CPU cpu0 pid=3 ticks=1 time=10_

Machine View
Finish Quantum ! cpu0 pid=7 ticks=3 time=30
Running on CPU cpu0 pid=3 ticks=0 time=0
Running on CPU cpu0 pid=3 ticks=1 time=10
Running on CPU cpu0 pid=3 ticks=2 time=20
Finish Quantum ! cpu0 pid=3 ticks=3 time=30
Running on CPU cpu0 pid=4 ticks=0 time=0
Running on CPU cpu0 pid=4 ticks=1 time=10
Running on CPU cpu0 pid=4 ticks=2 time=20
Finish Quantum ! cpu0 pid=4 ticks=3 time=30
Running on CPU cpu0 pid=5 ticks=0 time=0
Running on CPU cpu0 pid=5 ticks=1 time=10
Running on CPU cpu0 pid=5 ticks=2 time=20
Running on CPU cpu0 pid=5 ticks=2 time=20
Finish Quantum ! cpu0 pid=5 ticks=3 time=30
Running on CPU cpu0 pid=6 ticks=0 time=0
Running on CPU cpu0 pid=6 ticks=1 time=10
Running on CPU cpu0 pid=6 ticks=2 time=20
Finish Quantum ! cpu0 pid=6 ticks=3 time=30
Running on CPU cpu0 pid=7 ticks=0 time=0
Running on CPU cpu0 pid=7 ticks=1 time=10
Running on CPU cpu0 pid=7 ticks=2 time=20
Finish Quantum ! cpu0 pid=7 ticks=3 time=30
Running on CPU cpu0 pid=3 ticks=0 time=0
Running on CPU cpu0 pid=3 ticks=1 time=10_
```

برای حالت mp : در این حالت تعداد هسته ها را 3 تا قرار میدهیم ، یعنی 2 تا Ecore پس داریم :

در این حالت می بینیم الگوریتم باز هم به درستی کار میکند و زمان بندی هر دو cpu به صورت rr است با این تفاوت که مثلا اگر کار cpu0 با پردازش 5 تمام بشود و بخواید پردازش بعدی را شروع کند ، در حالت تکی ، 6 را شروع میکرد ولی اینجا ممکن است که خود پردازش 6 در حال اجرا توسط cpu2 باشد ، در این صورت cpu0 پردازش 7 را انتخاب میکند.بقیه موارد این 2 پردازنده از هم مستقل می باشد .هم چنین با توجه به lockهایی که صورت می گیرد 2 تا پردازنده با هم سر ptable درگیر نمیشوند .

[illegible]

بخش ۴.۳) هسته‌های P با شماره CPUID فرد: زمان‌بند اولین ورود-اولین رسیدگی

در ابتدای این بخش، یک فیلد جدید به نام `create_time` به `struct proc` (در فایل `proc.h`) اضافه می‌کنیم که زمان ایجاد هر پردازش را نگهداری می‌کند:

```
struct proc {
    uint sz;
    pde_t* pgdir;
    char *kstack;
    enum procstate state;
    int pid;
    struct proc *parent;
    struct trapframe *tf;
    struct context *context;
    void *chan;
    int killed;
    struct file *ofile[NOFILE];
    struct inode *cwd;
    char name[16];
    int priority;

    uint create_time;
};
```

سپس مقدار `create_time` هر پردازش را در هنگام ایجاد آن پردازش در تابع `allocproc` (در فایل `proc.c`) تعیین می‌کنیم. برای این کار در واقع مقدار `create_time` را برابر متغیر گلوبال `ticks` قرار می‌دهیم: (`ticks` یک متغیر گلوبال در هسته سیستم‌عامل است که با هر بار آمدن یک `timer interrupt`، یک واحد افزایش می‌یابد).

```
found:
    p->state = EMBRYO;
    p->pid = nextpid++;
    p->priority = 1;

    release(&ptable.lock);

    p->create_time = ticks; // add for FCFS
```

اکنون منطق اصلی این زمان‌بند را در تابع scheduler (در فایل proc.c) پیاده‌سازی می‌کنیم:

```
// start of FCFS
if (cpu_id % 2 == 1) {
    struct proc *best = 0;
    for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
        if (p->state != RUNNABLE)
            continue;
        if (best == 0 || p->create_time < best->create_time)
            best = p;
    }

    if (best) {
        c->proc = best;
        switchvm(best);
        best->state = RUNNING;

        switch(&(c->scheduler), best->context);

        switchkvm();
        c->proc = 0;
    }

    release(&ptable.lock);
    continue;
}
// end of FCFS
```

توضیح کد: مقدار `cpu_id` توسط تابع `cupid()` تعیین شده است. در صورتی که فرد باشد (یعنی از نوع P باشد)، الگوریتم زمان‌بند اجرا می‌شود. در این الگوریتم بر روی جدول پردازنده‌ها حلقه می‌زنیم و پردازنده‌ای که کوچکترین مقدار `create_time` را دارد را در متغیر `best` ذخیره می‌کنیم.

سپس در صورتی که پردازنده‌ای یافت شده بود، پردازنده را در فیلد `proc` پردازنده فعلی قرار می‌دهد، فضای آدرس سخت‌افزار را روی `pgdir` همین پردازنده تنظیم می‌کند، وضعیت پردازنده را به `RUNNING` تغییر می‌دهد و سپس `context switch` را توسط تابع `switch` انجام می‌دهد. این تابع تا زمانی که پردازنده روی `cpu` باشد برنمی‌گردد. اما زمانی که برگشت، یعنی دیگر پردازنده روی `cpu` نیست و بنابراین فضای آدرس سخت‌افزار روی کرنل برمی‌گردانیم و فیلد `proc` پردازنده را برابر 0 قرار می‌دهیم؛ چرا که فعلاً هیچ پردازنده‌ای در حال اجرا بر روی آن نیست.

برای آنکه بتوانیم عملکرد الگوریتم را بسنجیم، یک برنامه سطح کاربر می‌نویسیم که ۳ پردازش `fork` می‌کند. در ابتدا پردازش A را ایجاد می‌کند اما بلافاصله آن را در حالت `sleep` قرار می‌دهد تا در صف `cpu` نباشد. سپس B را ایجاد می‌کند و آن را هم در حالت `sleep` قرار می‌دهد. در آخر نیز C را ایجاد می‌کند و `cpu` را به کمک تابع `busy` درگیر اجرای این پردازش می‌کند. سپس در حین اجرای این پردازش، پردازش B بیدار می‌شود و `cpu` را درخواست می‌کند و به شکل مشابه در حین اجرای پردازش B، پردازش A بیدار می‌شود.

```
pidA = fork();
if(pidA < 0){
    exit();
}

if(pidA == 0){
    printf(1, "A: start (sleeping)\n");
    sleep(40);
    printf(1, "A: woke up\n");
    busy("A", 40000000);
    printf(1, "A: done\n");
    exit();
}

sleep(5);

pidB = fork();
if(pidB < 0){
    exit();
}

if(pidB == 0){
    printf(1, "B: start (sleeping)\n");
    sleep(20);
    printf(1, "B: woke up\n");
    busy("B", 50000000);
    printf(1, "B: done\n");
    exit();
}

sleep(5);

pidC = fork();
if(pidC < 0){
    exit();
}

if(pidC == 0){
    printf(1, "C: start (no sleep)\n");
    busy("C", 60000000);
    printf(1, "C: done\n");
    exit();
}
```

حال برای اینکه بتوانیم نتایج را تحلیل کنیم تعداد cpu را برابر ۱ قرار می‌دهیم و سیاست زمانبندی این cpu را نیز FCFS قرار می‌دهیم. همچنین یک cprintf در تابع trap قرار می‌دهیم تا pid پردازش‌های در حال اجرا را پرینت کنیم. سپس برنامه سطح کاربر را اجرا می‌کنیم. در صورتی که الگوریتم به درستی پیاده‌سازی شده باشد، باید در ابتدا cpu را به C بدهد. سپس زمانی که B بیدار شد، به دلیل آنکه زودتر از C ایجاد شده است، cpu را به B بدهد. و در زمانی که A بیدار شد نیز، چون از همه پردازش‌ها زودتر ایجاد شده است، cpu را به A بدهد.

نتیجه اجرا:

```
$ FCFS_test
fcfstest: start
A: start (sleeping)
B: start (sleeping)
C: start (no sleep)
C: busy start
tick 588: pid 6
tick 589: pid 6
tick 590: pid 6
tick 591: pid 6
tick 592: pid 6
tick 593: pid 6
```

```
B: woke up
B: busy start
tick 604: pid 5
tick 605: pid 5
tick 606: pid 5
tick 607: pid 5
tick 608: pid 5
tick 609: pid 5
```

```
A: woke up
A: busy start
tick 619: pid 4
tick 620: pid 4
tick 621: pid 4
tick 622: pid 4
tick 623: pid 4
tick 624: pid 4
tick 625: pid 4
```

```
A: busy end
A: done
tick 656: pid 4
tick 657: pid 5
tick 658: pid 5
tick 659: pid 5
tick 660: pid 5
tick 661: pid 5
```

```
B: busy end
B: done
tick 691: pid 6
tick 692: pid 6
tick 693: pid 6
tick 694: pid 6
tick 695: pid 6
tick 696: pid 6
```

```
tick 729: pid 6
tick 730: pid 6
tick 731: pid 6
tick 732: pid 6
tick 733: pid 6
tick 734: pid 6
C: busy end
C: done
fcfstest: done
```

همانطور که مشاهده می‌شود ترتیب اجرای پردازش‌ها به درستی رعایت شده است و در واقع در این الگوریتم، اولویت را به پردازش‌های می‌دهیم که زودتر ایجاد شده است.

بخش ۴.۴) جابجایی پردازش میان صف‌ها

در ابتدای این بخش یک فیلد به نام `host_cpu` به `struct proc` (در فایل `proc.h`) اضافه می‌کنیم. این فیلد، `id` پردازنده‌ای را نگه می‌دارد که یک پردازش باید بر روی آن اجرا شود. علت این کار این است که در این بخش، هر پردازش فقط به یک پردازنده تعلق می‌گیرد و فقط بر روی آن اجرا می‌شود. بنابراین باید چنین فیلدی را برای هر پردازش در نظر بگیریم و همچنین در تابع `scheduler` (در فایل `proc.c`)، فقط پردازش‌های را برای اجرا انتخاب می‌کنیم که مقدار `host_cpu` آن برابر با مقدار `cupid()` باشد:

```
struct proc {
    uint sz;
    pde_t* pgdir;
    char *kstack;
    enum procstate state;
    int pid;
    struct proc *parent;
    struct trapframe *tf;
    struct context *context;
    void *chan;
    int killed;
    struct file *ofile[NOFIL];
    struct inode *cwd;
    char name[16];
    int priority;

    uint create_time;
    int host_cpu;
};
```

سپس یک تابع کمکی به نام `find_least_ecore()` تعریف می‌کنیم که در بین تمام پردازنده‌های `ECORE` جستجو کرده و `id` پردازنده‌ای که کمترین تعداد پردازش را دارد برمیگرداند:

```

int find_least_ecore(void) {
    int i;
    int count[NCPU];
    int best = -1;
    int best_count = 0x7fffffff;
    struct proc *p;

    for(i = 0; i < ncpu; i++){
        if(cpus[i].type == ECORE)
            count[i] = 0;
        else
            count[i] = -1;
    }

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(!(p->state == RUNNABLE || p->state == RUNNING))
            continue;
        if(p->host_cpu < 0 || p->host_cpu >= ncpu)
            continue;
        if(cpus[p->host_cpu].type != ECORE)
            continue;

        count[p->host_cpu]++;
    }

    for(i = 0; i < ncpu; i++){
        if(count[i] < 0)
            continue;
        if(count[i] < best_count){
            best_count = count[i];
            best = i;
        }
    }

    release(&ptable.lock);

    if(best < 0)
        best = 0;

    return best;
}

```

سپس در تابع `userinit` (در فایل `proc.c`) مقدار `host_cpu` پردازش `init` را با استفاده از تابع `find_least_ecore()` مقداردهی می‌کنیم. علت این کار این است که در این بخش گفته شده است که پردازش `init` باید در صف یکی از هسته‌های `E` قرار بگیرد:

```

p->tf->cs = (SEG_UCODE << 3) | DPL_USER;
p->tf->ds = (SEG_UDATA << 3) | DPL_USER;
p->tf->es = p->tf->ds;
p->tf->ss = p->tf->ds;
p->tf->eflags = FL_IF;
p->tf->esp = PGSIZE;
p->tf->eip = 0; // beginning of initcode.S
p->host_cpu = find_least_ecore(); // add for moving between queues

```

سپس در تابع fork (در فایل proc.c) مقدار host_cpu پردازش‌های که fork می‌شود را باز هم با استفاده از تابع find_least_ecore() بدست می‌آوریم.

```
np->sz = curproc->sz;
np->parent = curproc;
*np->tf = *curproc->tf;
np->host_cpu = find_least_ecore(); // add for moving between queues
```

سپس در تابع trap (در فایل trap.c)، هر بار که وقفه تایمر بیاید، در صورتی که مقدار ticks بر ۵ بخش‌پذیر باشد، (یعنی هر ۵۰ میلی ثانیه یکبار) برای تمام هسته‌های ECORE تابع balance_queues() را اجرا می‌کنیم:

```
switch(tf->trapno){
case T_IRQ0 + IRQ_TIMER:
    if(cpuid() == 0){
        acquire(&tickslock);
        ticks++;
        wakeup(&ticks);
        release(&tickslock);
    }

    if (mycpu()->type == ECORE && ticks % 5 == 0) { // add for moving between queues
        balance_queues();
    }
}
```

در آخر نیز تابع balance_queues() را به فایل proc.c اضافه می‌کنیم:

```
void
balance_queues(void)
{
    int i;
    struct proc *p;
    int runnable_count[NCPU];
    int min_P_id = -1;
    int min_P_count = 0x7fffffff;
    int E_id = cpuid();
    int E_count = 0;

    acquire(&ptable.lock);

    for(i = 0; i < ncpu; i++) {
        if(cpus[i].type == PCORE)
            runnable_count[i] = 0;
        else
            runnable_count[i] = -1;
    }

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->host_cpu == E_id && p->state == RUNNABLE){
            E_count++;
        }
        if(p->host_cpu >= 0 && p->host_cpu < ncpu &&
            cpus[p->host_cpu].type == PCORE &&
            (p->state == RUNNABLE || p->state == RUNNING)){
            runnable_count[p->host_cpu]++;
        }
    }
}
```

توضیح کد: در حلقه اول کد، آرایه `runnable_count` را مقداردهی اولیه می‌کنیم. این آرایه که به طول تعداد `cpu` ها تشکیل می‌شود، به ازای `ECORE` ها مقدار 1- و به ازای `PCORE` ها مقدار 0 می‌گیرد.

در حلقه دوم، با پر کردن آرایه `runnable_count` تعداد پردازنده‌های مربوط به هر هسته `PCORE` را بدست می‌آوریم. همچنین با آوردن متغیر `E_count`، تعداد پردازنده‌های مربوط به هسته `ECORE` را بدست می‌آوریم. (همان هسته‌ای که این تابع را صدا زده است و ممکن است نیاز به بالانس داشته باشد).

```
for(i = 0; i < ncpu; i++){
    int cnt = runnable_count[i];
    if(cnt < 0)
        continue;
    if(cnt < min_P_count){
        min_P_count = cnt;
        min_P_id = i;
    }
}

if(min_P_id < 0 || E_count < min_P_count + 3){
    release(&table.lock);
    return;
}

struct proc *first = 0;
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state != RUNNABLE)
        continue;
    if(p->host_cpu != E_id)
        continue;
    if(p == initproc)
        continue;
    if(strncmp(p->name, "sh", 2) == 0)
        continue;
    if (first == 0 || p->create_time < first->create_time)
        first = p;
}

if (first) {
    cprintf("moving process %d from CPU %d to CPU %d\n", first->pid, E_id, min_P_id);
    first->host_cpu = min_P_id;
}

release(&table.lock);
}
```

توضیح کد: در حلقه سوم، مقدار مینیمم موجود در آرایه `runnable_count` را پیدا می‌کنیم. این مقدار را در متغیر `min_P_count` و `id` هسته‌ای که مینیمم پردازنده را دارد در متغیر `min_P_id` ذخیره می‌کنیم.

سپس در دستور `if`، در صورتی که تعداد پردازنده‌های `ECORE` از مینیمم تعداد پردازنده‌های `PCORE` ها حداقل ۳ تا بیشتر نباشد، کاری نمی‌کنیم.

در حلقه چهارم، اولین پردازنده‌ای که مربوط به پردازنده `ECORE` است را پیدا می‌کنیم (اولین پردازنده به جز `init` و `shell`) و آن را در `first` ذخیره می‌کنیم.

در آخر نیز مقدار `host_cpu` پردازنده یافت شده را به مقدار `min_P_id` تغییر می‌دهیم.

برای آنکه بتوانیم عملکرد تابع `balance_queues()` را بسنجیم، یک برنامه سطح کاربر می‌نویسیم که ۴ پردازنده `fork` می‌کند و هر کدام را به یک عملیات سنگین و طولانی روی `cpu` مشغول می‌کند: (پردازنده‌های `cpu bound` هستند).

```
#include "types.h"
#include "stat.h"
#include "user.h"

#define NCHILD 4

void busy(int id) {
    volatile int x = 0;
    printf(1, "child %d: start\n", id);
    for(int i = 0; i < 200000000; i++)
        x += i;
    printf(1, "child %d: done\n", id);
}

int main(void) {
    int i, pid;
    printf(1, "balancetest: start\n");

    for(i = 0; i < NCHILD; i++){
        pid = fork();
        if(pid < 0){
            exit();
        }

        if(pid == 0){
            busy(i);
            exit();
        }
    }

    sleep(10);

    for(i = 0; i < NCHILD; i++)
        wait();

    printf(1, "balancetest: done\n");
    exit();
}
```

سپس این برنامه را یکبار بدون استفاده از `balance_queues()` (چند خط اضافه شده به تابع `trap` را کامنت می‌کنیم) و یکبار با استفاده از `balanced_queues()` اجرا می‌کنیم. (برای آنکه تحلیل نتایج آسان‌تر شود سیستم را با دو `cpu` اجرا می‌کنیم و همچنین سیاست زمانبندی تمام `cpu` ها را `FCFS` قرار می‌دهیم).

نتیجه اجرا بدون استفاده از بالانس:

```
$ balance_test
balancetest: start
child 0: start
child 0: done
child 1: start
child 1: done
child 2: start
child 2: done
child 3: start
child 3: done
balancetest: done
$
```

در این حالت همانطور که مشاهده می‌شود تمام پردازها به ترتیب اجرا می‌شوند و هیچ همپوشانی نداریم. در واقع در این حالت تمام پردازها روی `cpu0` اجرا می‌شوند و چون سیاست زمانبند، `FCFS` است تمام پردازها به ترتیب اجرا می‌شوند.

نتیجه اجرا با استفاده از بالانس:

```
$ balance_test
balancetest: start
child 0: start
moving process 3 from CPU 0 to CPU 1
moving process 5 from CPU 0 to CPU 1
child 1: start
child 0: done
child 2: start
child 1: done
child 2: done
child 3: start
child 3: done
balancetest: done
$ _
```

همانطور که مشاهده می‌شود، دو پردازهی مختلف (با شناسه‌های ۳ و ۵) از صف پردازهای `cpu0` به صف `cpu1` منتقل شده‌اند. این جابه‌جایی‌ها دقیقاً زمانی انجام می‌شود که طبق الگوریتم، تعداد پردازهای آماده‌ی روی `cpu0` حداقل سه واحد بیشتر از سبک‌ترین هسته‌ی `P`، (اینجا `cpu1`) می‌شود.

بنابراین پردازها به طور همزمان روی دو `cpu` اجرا می‌شوند و همپوشانی به وجود می‌آید. مثلاً قبل آنکه `child0` تمام بشود، `child1` روی `cpu` دیگر شروع به اجرا می‌کند. یا مثلاً قبل از آنکه `child1` تمام شود، `child2` روی `cpu` دیگر شروع به اجرا می‌کند. پس در واقع تابع `balance_queues()` به درستی کار می‌کند.

بخش ۵.۵) فراخوانی های سیستمی مورد نیاز:

ابتدا در فایل syscall.h متغیر های مربوطه را define میکنیم (شماره جدید برای سیستم کال های جدید)

```
#define SYS_start_measure 27
#define SYS_end_measure 28
#define SYS_print_info 29
```

سپس در فایل user.h پروتوتایپ توابع را مینویسیم:

```
int start_measure(void);
int end_measure(void);
int print_info(void);
```

حالا در فایل usys.S خطوط زیر را باید اضافه کنیم (رابط بین کاربر و کرنل)

```
SYSCALL(start_measure)
SYSCALL(end_measure)
SYSCALL(print_info)
```

متغیر هایی که نیاز داریم برای سیستم کال ها جدید مان در استراکچر مربوط به هر پردازش از قبل تعریف کرده ایم:

```
uint create_time;
int host_cpu;
int ticks;
```

حالا در فایل proc.c متغیر های گلوبال زیر را تعریف میکنیم که به ما کمک میکنند که زمان شروع اندازه گیری تعداد پردازش های تمام شده و یک فلگ مربوط به اینکه آیا در حال اندازه گیری هستیم یا نه را نگه داریم.

```
int measurement_start_ticks = 0; // added for start time
int finished_process_count = 0;
int measurement_active = 0; //flag for determining whether we are measuring
```

در مراحل قبل زمان شروع پردازش ذخیره میشود (اضافه شده است) و نیازی نیست که ما این موضوع را اضافه کنیم (در تابع allocproc)

```
p->create_time = ticks; // add for FCFS
```

برای محاسبه گذردهی باید بدانیم در زمان مربوطه چند پردازش کارشان تمام شده است لذا باید در آخر تابع exit قبل از اینکه پردازش به حالت zombie بدل شود تعداد پردازش های کل انجام شده را (متغیر گلوبال) را یک واحد افزایش دهیم (در صورتی که فلگ در حال اندازه گیری ۱ بود)

```
if(measurement_active) {  
    finished_process_count++;  
}
```

حالا نوبت به پیاده سازی منطق سیستم کال ها میرسد. در فایل sysproc.c ابتدا متغیر های گلوبالی که پیشتر تعریف کرده بودیم را وارد میکنیم تا از آن استفاده کنیم:

```
extern int measurement_start_ticks;  
extern int finished_process_count;  
extern int measurement_active;
```

حالا باید توابع را یکی یکی تعریف کنیم:

تابع sys_start_measure() باید فلگ اندازه گیری را فعال (۱) بکند سپس متغیر finished_process_count را ریست کند (زیرا در حال شروع اندازه گیری هستیم) و مقدار شروع ticks اندازه گیری (متغیر گلوبال) را برابر ticks فعلی سیستم قرار بدهد.

```
int sys_start_measure(void) {  
    measurement_active = 1;  
    finished_process_count = 0;  
    measurement_start_ticks = ticks;  
    cprintf("Measurement Started at tick %d\n", measurement_start_ticks);  
    return 0;  
}
```

تابع `sys_end_measure()` باید پایان اندازه گیری را مشخص کرده و مقدار گذردهی را محاسبه و چاپ نماید. ابتدا چک باید کند که فلگ اندازه گیری فعال باشد و اگر نبود خطا چاپ کند و برگردد زیرا فلگ تنها زمانی فعال میشود که جایی سیستم کال `start_measure` فراخوانی شده باشد و اگر نشده باشد فراخوانی سیستم کال پایان اندازه گیری بی معناست. سپس زمان کل این فرآیند اندازه گیری باید محاسبه شود که برابر است با تفاضل مقدار `ticks` فعلی با مقدار اولیه آن که در متغیر گلوبال ذخیره شده است.

چون در اینجا `float` نداریم باید مقدار صحیح و اعشاری را از هم جدا محاسبه کرده و نمایش دهیم:

```
int sys_end_measure(void) {
    if (!measurement_active) {
        cprintf("Error: Measurement was not started.\n");
        return -1;
    }

    int current_ticks = ticks;
    int duration = current_ticks - measurement_start_ticks;

    if (duration == 0) duration = 1;

    int throughput_integer = finished_process_count / duration;
    int throughput_fraction = ((finished_process_count * 100) / duration) % 100;

    cprintf("\n--- Measurement Results ---\n");
    cprintf("Total Time (ticks): %d\n", duration);
    cprintf("Finished Processes: %d\n", finished_process_count);
    if(throughput_fraction < 10)
        cprintf("Throughput: %d.0%d processes/tick\n", throughput_integer, throughput_fraction);
    else
        cprintf("Throughput: %d.%d processes/tick\n", throughput_integer, throughput_fraction);
    measurement_active = 0;
    return 0;
}
```

و در آخر فلگ را غیرفعال میکنیم.

برای تابع `sys_print_info()` ابتدا باید محتوای آن پردازش را نگه داشت (اطلاعات مربوط به آن) و مقدار `lifetime` آن را محاسبه کرد از طریق کم کردن `ticks` فعلی از `create_time` پردازش. سپس به راحتی میتوان بصورت زیر اطلاعات مربوط به پردازش را چاپ کرد (اگر پردازنده ای که این پردازش روی آن اجرا میشود شماره زوج داشته باشد E-CORE است و اگر فرد باشد P-CORE خواهد بود)

```

int sys_print_info(void) {
    struct proc *p = myproc();
    int lifetime = ticks - p->create_time;

    cprintf("\n--- Process Info ---\n");
    cprintf("Name: %s\n", p->name);
    cprintf("PID: %d\n", p->pid);
    cprintf("Lifetime: %d ticks\n", lifetime);

    int core_id = p->host_cpu;
    if (core_id % 2 == 0) {
        cprintf("Algorithm: Round Robin (E-Core)\n");
    } else {
        cprintf("Algorithm: Modified FCFS (P-Core)\n");
    }

    return 0;
}

```

در نهایت در فایل `syscall.c` پروتوتایپ توابع نوشته شده را اضافه میکنیم و آنها را (به شماره ای که در ابتدا به آنها اختصاص داده ایم) مپ میکنیم:

```

extern int sys_start_measure(void);
extern int sys_end_measure(void);
extern int sys_print_info(void);

```

```

[SYS_start_measure] sys_start_measure,
[SYS_end_measure]   sys_end_measure,
[SYS_print_info]    sys_print_info,

```

بخش ۵.۶) برنامه های سطح کاربر

یک فایل جدید به نام `test_sched.c` ایجاد میکنیم. نام فایل را ابتدا در میک فایل در قسمت مربوطه اضافه میکنیم تا بعنوان یک دستور شناخته شود.

کد داخل این فایل باید شامل دو بخش باشد. یکی بخش اصلی که چندین پردازش تولید میکند (با استفاده از دستور `fork()`) و یکی دیگر هم بخشی که یک تابع با حجم محاسبات بالا به ما ارائه میدهد. در ابتدا بخش اصلی (تابع `main`) باید سیستم کال `start_measure()` را فراخوانی کند تا اندازه گیری ما شروع شود. سپس به تعداد ۶ فرزند با `fork()` میسازد و اگر دچار خطا شد خطا را چاپ و برمیگردد و اگر نشد فرزند تابع پردازش سنگین را اجرا میکند سپس `print_info()` را صدا میزند که سیستم کالی است که در بخش قبل نوشته ایم. حالا `exit()` میکند (با توجه به تغییری که در این سیستم کال داده ایم باید تعداد پردازش های تمام شده به درستی افزایش یابد) والد به تعداد فرزند هایش صبر میکند تا کار آنها تمام شوند سپس `print_info()` را صدا میزند تا اطلاعات مربوط به خودش نیز نمایش داده شود و در نهایت `end_measure()` را صدا میزند که اندازه گیری برای استفاده های بعدی دچار خطا نشود و در نهایت `exit()` میکند.

```
void heavy_computation() {
    volatile double x = 0;
    for (double z = 0; z < 4000000.0; z += 1.0) {
        x = x + 3.14 * 89.64;
    }
}

int main(int argc, char *argv[]) {
    printf(1, "Starting scheduler test\n");

    start_measure();
    int N_CHILDREN = 6;

    for (int i = 0; i < N_CHILDREN; i++) {
        int pid = fork();

        if (pid < 0) {
            printf(1, "Fork failed\n");
            exit();
        }

        if (pid == 0) {
            heavy_computation();
            print_info();
            exit();
        }
    }

    for (int i = 0; i < N_CHILDREN; i++) {
        wait();
    }
    print_info();
    end_measure();
    exit();
}
```

```
$ test_sched
Starting scheduler test
Measurement Started at tick 430
moving process 4 from CPU 0 to CPU 1
moving process 5 from CPU 0 to CPU 1
```

```
--- Process Info ---
Name: test_sched
PID: 4
Lifetime: 46 ticks
Algorithm: Modified FCFS (P-Core)
```

```
--- Process Info ---
Name: test_sched
PID: 9
Lifetime: 57 ticks
Algorithm: Round Robin (E-Core)
```

```
--- Process Info ---
Name: test_sched
PID: 5
Lifetime: 87 ticks
Algorithm: Modified FCFS (P-Core)
moving process 3 from CPU 0 to CPU 1
```

```
--- Process Info ---
Name: test_sched
PID: 7
Lifetime: 98 ticks
Algorithm: Round Robin (E-Core)
```

```
--- Process Info ---
Name: test_sched
PID: 6
Lifetime: 139 ticks
Algorithm: Round Robin (E-Core)
```

```
--- Process Info ---
Name: test_sched
PID: 8
Lifetime: 179 ticks
Algorithm: Round Robin (E-Core)
```

```
--- Process Info ---
Name: test_sched
PID: 3
Lifetime: 183 ticks
Algorithm: Modified FCFS (P-Core)
```

```
--- Measurement Results ---
Total Time (ticks): 183
Finished Processes: 6
Throughput: 0.03 processes/tick
```

با توجه به اینکه اگر تعداد پردازه ها روی هسته زوج ۳ تا بیشتر از هسته فرد باشد (در حالت ۲ هسته ای) یک پردازه به هسته فرد می‌رود و در اینجا ۷ پردازه داریم میبینیم که ابتدا ۲ پردازه به هسته فرد و سپس هسته والد هم به هسته فرد می‌روند (با اجرا شدن دومین پردازه روی هسته فرد بعثت سریعتر بودن^۱ تفاضل مجددا بیش از ۳ شده برای همین پردازه والد هم به هسته فرد می‌رود)

در عکس بالا مشاهده میشود که پردازه ۴ و ۹ تقریبا در یک بازه زمانی نزدیک به هم کارشان را تمام کرده اند (البته چون هسته فرد سریعتر هست یک مقدار زودتر انجام میدهد) همچنین ۵ و ۷ هم در بازه نزدیک به هم کارشان تمام شده است که نشان از این موضوع است که این ۴ تا دو به دو روی پردازنده های زوج و فرد بصورت موازی در حال اجرا شدن بوده اند. پردازه والد که به هسته فرد منتقل میشود باید تا آخرین فرزندش که کارش تمام شود صبر کند لذا `print_info()` نمیکند. پردازه های ۷ و ۶ و ۸ هم که روی پردازنده زوج بودند به ترتیب اجرا شده و سپس پردازه والد خاتمه می یابد.

مقدار گذردهی که برابر تعداد پردازه ها تقسیم بر زمان کل است نیز برابر $6/183$ خواهد بود (پردازه والد حساب نمیشود) که تقریبا معادل همان 0.03 است (در صورت تقریب زدن تا ۲ اعشار) که نشان از درستی محاسبات مان دارد.